In-Memory Logic Synthesis Methods based on Read Decoupled 8T and Transpose 11T SRAMs

Feng Wei, Xiaole Cui*, Sunrui Zhang
Key Lab of Integrated Microsystems
Peking University Shenzhen Graduate School, Shenzhen 518055, China
Email: cuixl@pkusz.edu.cn, weifeng94@sina.com, alwayswry@stu.pku.edu.cn

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11043683

***Abstract*—In-memory computing (IMC) is regarded as the promising computer architecture to break through the von-Neumann bottleneck. Some in-memory logic operations, such as *AND/NAND/OR/NOR/XOR*, have been proposed to be implemented within SRAM array. However, the studies on the SRAM-based in-memory logic operations still focus on the gate-level. The implementation method of arbitrary logic functions is an open issue till now. In order to address this problem, this paper proposes the general in-memory logic synthesis methods based on the read decoupled 8T SRAM array and the transpose 11T SRAM array. The proposed synthesis methods are applied to some MCNC benchmark circuits. The synthesis results show that the in-memory logic circuit generated by the 8T SRAM array consumes smaller sub-array size, and the in-memory logic circuit generated by the 11T SRAM array consumes less compute cycles.**

Keywords—In-memory computing, SRAM, logic synthesis, general computing platform

I. Introduction

The traditional von-Neumann computer architecture separates the memory and the processing units. It limits the throughput of the data path, and consumes large amount of energy and latency on the data transportation. The in-memory computing (IMC) architecture, which is regarded as a promising approach to address the challenge of the von-Neumann bottleneck, becomes more and more popular in recent years. The IMC circuit implements the multiply-and-accumulate (MAC) or the logic operations within the memory array. The power and performance are improved because the IMC eliminates the extra data transfers between the memory and the processing units.

The SRAM-based MAC circuits have good performance, and many solutions were proposed and discussed in [1], [2], [3], [4], [5], [6], [7], [8]. Besides the in-memory MAC circuits, the in-memory logic operations are proposed. Till now, researchers have reported several SRAM circuits to implement in-memory logic operations. S. Jeloka et al. [9] implements the in-memory logic operations such as the *AND* and *NOR* using the standard push-rule 6T SRAM cells, which consumes the relatively small area, but it suffers from the low computation stability in multi-input logic operations. A. Agrawal [10] uses the 8T read decoupled SRAM cells to enable the column-wise in-memory *NAND*, *NOR*, *XOR* operations, and it supports the stable multi-input logic operations. Wei et al [11] proposes the 11T transpose SRAM cells to support the dual-direction in-memory logic operations, which also supports the stable multi-input logic operations. Some other implementation of in-memory logic operations are also proposed [12], [13], [14], [15]. These basic in-memory logic operations are applied to some specific in-memory application scenarios, e.g., the in-memory BIST circuit [16], which eliminates the conventional BIST circuit outside the SRAM array and significantly reduces the circuit area. However, the current studies on SRAM-based IMC logic operations focus on gate-level, the in-memory implementation methods of arbitrary logic functions is still an open issue.

This paper proposes the in-memory logic synthesis method to make the IMC as a general computing platform. The proposed synthesis methods are based on the read decoupled SRAM arrays, e.g., the 8T SRAM and 11T SRAM arrays. The remainder of the paper is organized as follows. Section II proposes the in-memory logic synthesis method based on the 8T SRAM array, which implements the column-wise logic operations. Section III proposes the in-memory logic synthesis method based on the 11T SRAM array, which implements the dual-direction logic operations. The synthesis results are discussed and compared in Section IV. Finally, the conclusion is drawn in Section V.

II. In-Memory Logic Synthesis based on 8T SRAM Array

A. The proposed 8T SRAM array

The read decoupled 8T SRAM cell can implement the stable multi-input logic operations. Fig.1 shows the 8T SRAM cell and its logic operations. The two-input *OR/NOR* operation is implemented by activating the two read word lines (RWLs) simultaneously. The *OR/NOR* operation can be extended to multi-input cases, by activating the multiple RWLs simultaneously. The *read/NOT* operation is implemented by the one-input *OR/NOR* operation. Fig.2 presents the proposed 8T SRAM array. It can implement the read-compute-store (RCS) function, which is introduced in [10]. The proposed 8T SRAM array enables a column-wise *OR/NOR* logic operation and writes the result of the logic operation back to a target cell in one cycle by simultaneously activating RWLs of the input cells and WWL of the target cell. The target cell of the write operation is in the same column but different row of the column-wise *OR/NOR* operation.

B. In-memory logic synthesis based on the proposed 8T SRAM array

The proposed 8T SRAM array can be used as a general computation platform, because the *NOR* operation is logic complete. For any logic function with the standard SOP (Sum of Products) form, it can be presented with a two-level *NOR-OR* expression by the De Morgan's law. Under this premise, the synthesis of arbitrary logic function is proposed with the following steps. The logic function $F1 = ABC + AB\overline{D} + BD$ is used as an example to illustrate the synthesis process, and the generated circuit is presented in Fig. 3. The synthesis process includes two phases: the mapping phase and the computing phase.

This work was supported by the National Natural Science Foundation under Grant No. 92373206, Shenzhen Science and Technology Program under Grant No. JCYJ20220818100814033, SGDX20230116093303006, KJZD20231023100201003 and KQTD20200820113105004.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:19 UTC from IEEE Xplore. Restrictions apply.

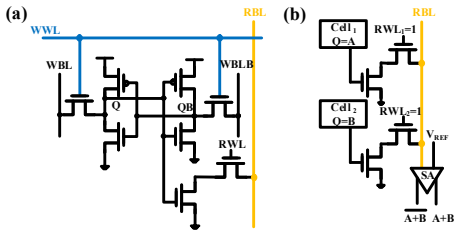


Fig.1 The read decoupled 8T SRAM cell (a) schematic [10] (b) column-wise *OR/NOR* operation

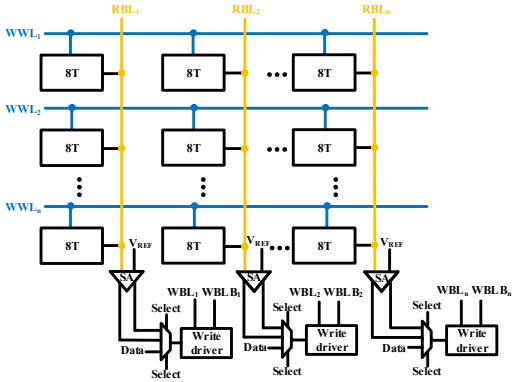


Fig.2 The proposed 8T SRAM array.

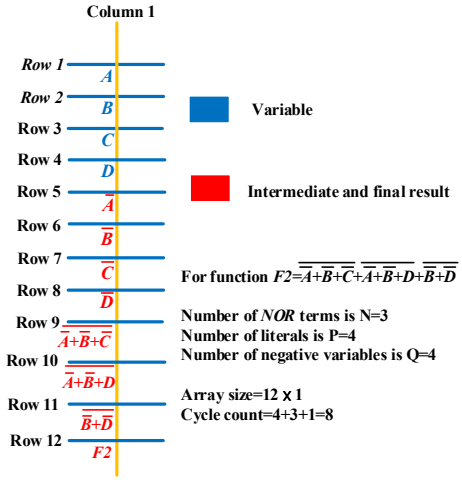


Fig.3 The implementation of *F1* based on 8T SRAM array (Each intersection represents an 8T SRAM cell).

Mapping phase:

Step 1: Transform the SOP form function *F1* into the *NOR-OR* expression *F2*. For example, the logic function *F1* above is transformed into $F2 = \overline{\bar{A} + \bar{B} + \bar{C}} + \overline{\bar{A} + \bar{B} + D} + \overline{\bar{B} + \bar{D}}$.

Step 2: Map the literals in *F2* to one unoccupied column, according to the *NOR-OR* expression *F2*.

Computing phase:

Step 1: Conduct the column-wise *NOT* operations to get the intermediate negative variable results one by one, and the results are stored in the new cells of the corresponding column in the same cycle. Since all the *NOR* terms are in the same column, different *NOR* terms that have the same negative variables share the intermediate negative variable results. In each cycle, only one RWL is activated to conduct a *NOT* operation, and one WWL is activated to write and store the result. For a *NOR-OR* function *F2* which has P literals and Q of them have negative variables, it takes Q cycles to get all the intermediate negative variable results in *F2*. As shown in Fig. 3, the RWLs of row1-4 are activated to conduct the column-wise *NOT* operation respectively, and the WWLs of row5-8 are activated to store the results one by one.

Step 2: Conduct the column-wise *NOR* operation to get the intermediate *NOR* term results one by one, and store the results in the new cells of the column. In this step, it takes N cycle to get N intermediate *NOR* term results. In the example of Fig. 3, the RWLs of row5-7 are simultaneously activated to obtain the value of the *NOR* term $\overline{\bar{A} + \bar{B} + \bar{C}}$, the RWLs of row4-6 are simultaneously activated to obtain the value of the *NOR* term $\overline{\bar{A} + \bar{B} + D}$, and the RWLs of row6 and row8 are simultaneously activated to obtain the value of the *NOR* term $\overline{\bar{B} + \bar{D}}$. The intermediate *NOR* term results are stored in row9-11 in the same column.

Step 3: Conduct the last column-wise *OR* operation to get the final result, and store the result to a new cell of the column. In Fig. 3, the RWLs of row9-11 are simultaneously activated to conduct the *OR* operation, and WWL of column12 is simultaneously activated to store the final result.

The area of the circuit generated by the proposed in-memory synthesis process is represented by the occupied SRAM sub-array, and the circuit performance is represented by the compute cycle counts. The proposed in-memory logic synthesis is able to predict the circuit area and cycle consumption accurately. Assume that the *NOR-OR* expression *F2* has N *NOR* terms, the number of literals in *F2* is P, and Q of them have negative variables. The size of the generated circuit is computed as formula (1).

$$\text{Sub-array size} = (P + Q + N + 1) \times 1 \quad (1)$$

The cycle count is calculated as formula (2).

$$Cycle\ count = Q + N + 1 \quad (2)$$

III. In-memory Logic Synthesis Based on Transpose 11T SRAM Array

A. The proposed 11T SRAM array

In order to implement the parallel computation during the in-memory logic synthesis, the 11T transpose SRAM cell is adopted, which is derived from [11]. Fig. 4 presents the schematic of the 11T SRAM cell and its logic operations. By simultaneously activating multiple horizontal read word lines (HRWLs) or vertical read word lines (VRWLs), the 11T SRAM cell implement the stable multi-input column-wise *OR/NOR* operations or row-wise *OR/NOR* operations. Fig. 5 presents the proposed 11T SRAM array. It can implement the read-compute-store (RCS) function both in the column-wise and the row-wise style.

B. In-memory logic synthesis based on the proposed 11T SRAM array

The logic function $F1 = ABC + AB\bar{D} + BD$ is also used as an example to illustrate the synthesis process based on the proposed 11T SRAM array, and the generated circuit is presented in Fig. 6. The synthesis process includes two phases: the mapping phase and the computing phase.

Mapping phase:

Step 1: Transform the SOP form function *F1* into the *NOR-OR* form *F2*. For example, the logic function *F1* above

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:19 UTC from IEEE Xplore. Restrictions apply.

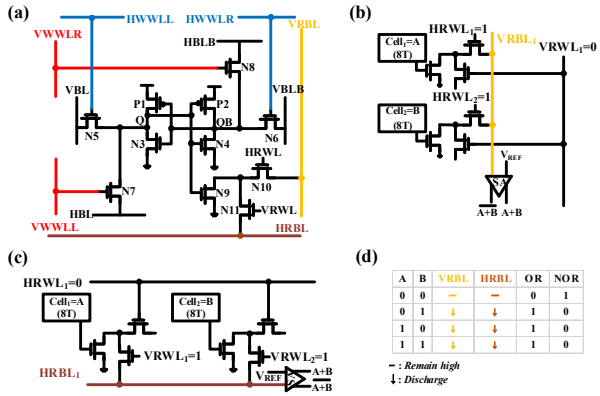


Fig.4 The transpose 11T SRAM cell (a) schematic [11] (b) column-wise *OR/NOR* operation (c) row-wise *OR/NOR* operation (d) true table of the dual-direction *OR/NOR* operation.

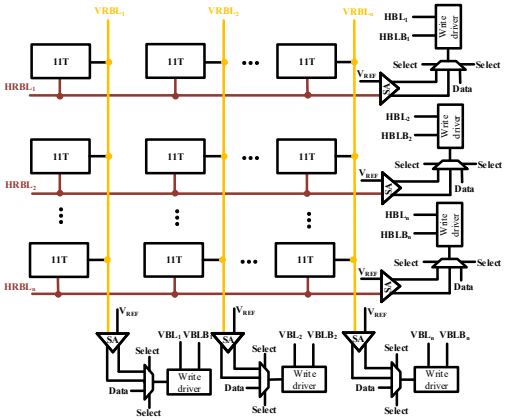


Fig.5 The proposed 11T SRAM array.

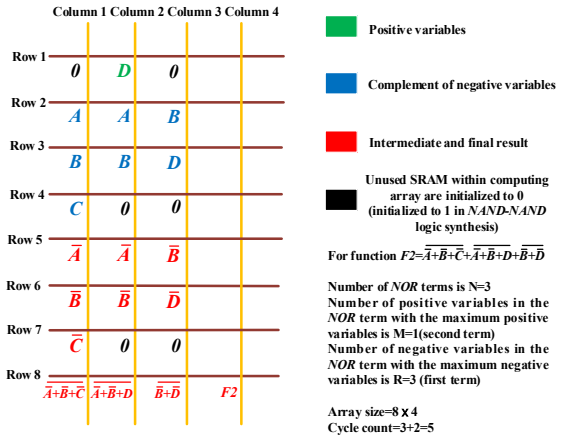


Fig.6 The implementation of *F1* based on 11T SRAM array (Each intersection represents an 11T SRAM cell).

is transformed into $F2 = \overline{\bar{A}+\bar{B}+\bar{C}} + \overline{\bar{A}+\bar{B}+D} + \overline{\bar{B}+\bar{D}}$.

Step 2: According to *NOR-OR* expression *F2*, map the positive variables in each *NOR* term to one unoccupied column, respectively. N columns are required for N *NOR* terms, and M rows are required if the number of positive variables in the *NOR* term with the maximum positive variables equals M. In Fig.6, three columns are occupied because there are three *NOR* terms in *F2*, and only the row1 is occupied because there is only one positive variable *D* in the second *NOR* terms.

Step 3: According to *NOR-OR* expression *F2*, map the complement of the negative variables in each *NOR* term to the cells at the same column occupied by the corresponding *NOR* term but at the different rows, respectively. R extra rows are required if the number of negative variables in the *NOR* term with the maximum negative variables equals R. In Fig.6, the row2-4 are occupied because the first *NOR* term $\bar{A}+\bar{B}+\bar{C}$ in *F2*, which contains the maximum number of negative variables, has three negative variables.

The unoccupied cells in the occupied columns and rows are initialized to 0, in order to utilize the parallel computing feature of the SRAM array.

Computing phase:

Step 1: Conduct the parallel column-wise *NOT* operation to get the intermediate negative variables, and store these results to the cells at the same column of the corresponding positive variables but at the different unoccupied rows in the same cycle. The column-wise *NOT* operation is implemented by the column-wise *NOR* operation, if only one corresponding read word line HRWL is activated. And one HWWL is activated to write and store the results in the same cycle. R cycles are consumed to get all the intermediate negative variables, if the number of negative variables in the *NOR* term with the maximum negative variables equals R. For *F2*, three cycles are consumed because R = 3. As shown in Fig. 6, the HRWLs of row2-4 are activated respectively to perform the parallel column-wise *NOT* operations, and the HWWLs of row5-7 are activated respectively to store the results.

Step 2: Conduct the parallel column-wise *NOR* operation to get the intermediate *NOR* term results, and store the results in a new row in the same cycle. In this step, it only takes one cycle to get all the intermediate *NOR* term results, because the *NOR* terms in different columns can be computed and stored in parallel. In the example of Fig. 6, the HRWLs of row1 and row5-7 are activated simultaneously to conduct the parallel column-wise *NOR* operation, and the HWWL of row8 is activated to store the results.

Step 3: Conduct the row-wise *OR* operation to get the final result, and store the result to a new cell in the same row. In Fig. 6, the VRWL of column1-3 are simultaneously activated to conduct the row-wise *OR* operation, and the VWWLs of column4 are activated at the same cycle to store the final result.

Assume that the *NOR-OR* expression of the logic function has N *NOR* terms, the number of negative variables in the *NOR* term with the maximum negative variables equals R, and the number of positive variables in the *NOR* term with the maximum positive variable equals M. In the mapping phase, it needs M rows to map the positive variables, R rows to map the complement of negative variables, R more rows are used to store the intermediate negative variable results, and one extra row to store the intermediate *NOR* term results. N columns are demanded to map N *NOR* terms, and one extra column is used to store the final result. The circuit area is calculated as formula (3).

$$Sub\text{-}array\ size = (M + 2R + 1) \times (N + 1) \qquad (3)$$

In the computing phase, R cycles of parallel column-wise *NOT* operation are consumed to compute and store all the negative variables in step 1, one cycle of parallel column-wise *NOR* operation is used to get all the *NOR* term results, and one extra cycle is used to conduct the row-wise *OR* operation and get the final result. The total cycle count is

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE COMPARISON OF THE SYNTHESIS RESULTS OF SOME *MCNC* BENCHMARK CIRCUITS GENERATED BY THE PROPOSED 8T AND 11T SRAM ARRAY

benchmark	Input number	*NOR-OR* type				*NAND-NAND* type			
		Transpose 11T SRAM		8T decoupled SRAM		Transpose 11T SRAM		8T decoupled SRAM	
		Array size, number of bitcells	Cycle count	Array size, number of bitcells	Cycle count	Array size, number of bitcells	Cycle count	Array size, number of bitcells	Cycle count
rd53f1	5	9 x 6 (6 x 9), 54	6	16 x 1, 16	11	5 x 6 (6 x 5), 30	2	11 x 1, 11	6
rd53f2	5	15 x 17 (17 x 15), 255	7	27 x 1, 27	22	14 x 17 (17 x 14), 238	6	27 x 1, 27	22
rd53f3	5	7 x 12 (12 x 7), 84	4	22 x 1, 22	17	7 x 12 (12 x 7), 84	4	22 x 1, 22	17
rd73f1	7	17 x 43 (43 x 17), 731	8	57 x 1, 57	50	15 x 43 (43 x 15), 645	6	57 x 1, 57	50
rd73f2	7	21 x 65 (65 x 21), 1365	9	79 x 1, 79	72	20 x 65 (65 x 20), 1300	8	79 x 1, 79	72
rd73f3	7	9 x 36 (36 x 9), 324	6	50 x 1, 50	43	5 x 36 (36 x 5), 180	2	43 x 1, 43	36
sao2f1	10	19 x 11 (11 x 19), 209	7	29 x 1, 29	19	22 x 11 (11 x 22), 242	10	31 x 1, 31	21
sao2f2	10	24 x 21 (21 x 24), 504	9	39 x 1, 39	29	26 x 21 (21 x 26), 546	11	41 x 1, 41	31
sao2f3	10	7 x 23 (23 x 7), 161	3	40 x 1, 40	30	10 x 23 (23 x 10), 230	6	43 x 1, 43	33
sao2f4	10	15 x 23 (23 x 15), 345	5	45 x 1, 45	35	20 x 27 (27 x 20), 540	10	47 x 1, 47	37

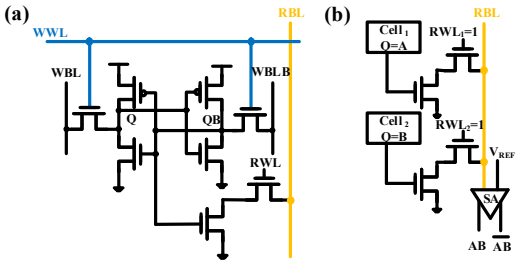


Fig.7 The 8T SRAM cell for *NAND-NAND* synthesis (a) schematic (b) column-wise *AND/NAND* operation.

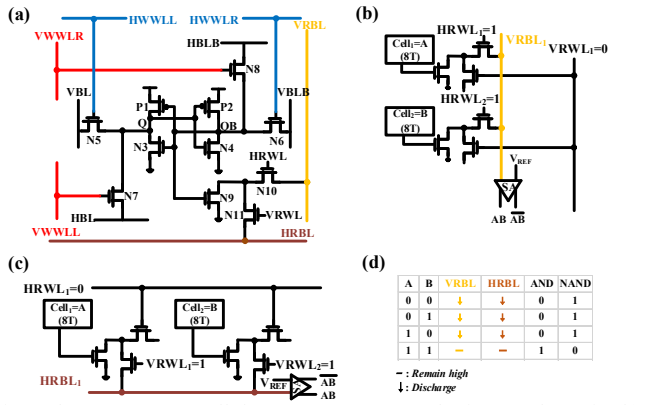


Fig.8 The 11T SRAM cell for *NAND-NAND* synthesis (a) schematic (b) column-wise *AND/NAND* operation (c) row-wise *AND/NAND* operation (d) true table of the dual-direction *AND/NAND* operation.

computed as formula (4).

$$Cycle\ count = R + 2 \tag{4}$$

IV. DISCUSSION AND COMPARISON

A. Comparisons of different in-memory logic Synthesis

The proposed in-memory logic synthesis processes are applied to some *MCNC* benchmark circuits, and the synthesis results are presented in Table I. The circuits generated by the logic synthesis based on the proposed 8T SRAM array and that based on the proposed 11T SRAM array are compared. In Table I, the size of the occupied sub-array and the number of bitcells represent the circuit area, the cycle count represents the circuit performance. Table I shows that the circuits generated by the synthesis process based on the proposed 8T SRAM array consumes less area than that of the proposed 11T SRAM array. However, the logic synthesis based on the proposed 11T SRAM array has the better performance than that of the 8T SRAM array. The reason is that the 8T SRAM array can only implement the column-wise logic operations, whereas the 11T SRAM array is able to take the advantage of the parallel computing feature, for it supports both the column-wise and the row-wise operations.

B. Array transpose synthesis of the proposed 11T SRAM array

In the proposed in-memory logic synthesis of the proposed 11T SRAM array, the different *NOR* terms are map into the different columns, then the column-wise *NOT* and *NOR* operations are implemented to get the intermediate *NOR* term results, and a row-wise *OR* operation is conducted at last to get the final result. However, with the proposed 11T SRAM array, different *NOR* terms can also be mapped into different rows, then the row-wise *NOT* and *NOR* operations are conducted, the column-wise *OR* operation is finally implemented. In this way, a transpose array is generated. In the synthesis results, the sub-array sizes occupied by the generated circuits are presented in the parentheses in Table I.

C. NAND-NAND logic synthesis

The *NAND-NAND* synthesis method can also implement the arbitrary logic functions in the SRAM array. Therefore, another 8T and 11T SRAM cells, which implements *NAND/AND* logic operation, are proposed in Fig. 7 and Fig.8. The proposed *AND/NAND* 8T and 11T SRAM cells can be used in the *NAND-NAND* logic synthesis. The mapping phase and computing phase of the *NAND-NAND* synthesis method are similar as that of *NOR-OR* synthesis method. The synthesis results are also presented in Table I. It can be seen that, both the area and cycle counts from the *NAND-NAND* synthesis method decrease for the logic function with less negative variable in the *NAND-NAND* expression, compared with that from the *NOR-OR* synthesis method.

V. CONCLUSION

This paper proposes the general in-memory logic synthesis methods based on the 8T SRAM and 11T SRAM arrays. Four specific synthesis methods considering the *NOR-OR* vs. *NAND-NAND* logic presentations and the array transpose function are introduced. These synthesis methods provide the flexible alternatives to implement the same logic function for the different sizes of the target SRAM array. The circuits generated by the proposed logic synthesis based on 8T SRAM array consumes less circuit area, whereas the circuit generated by the proposed logic synthesis based on 11T SRAM array has better performance.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:19 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] X. Qiao et al., "A 16.38TOPS and 4.55POPS/W SRAM Computing-in-Memory Macro for Signed Operands Computation and Batch Normalization Implementation," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 4, pp. 1706-1718, April 2024, doi: 10.1109/TCSI.2024.3353464.

[2] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An Energy-Efficient SRAM With In-Memory Dot-Product Computation for Low-Power Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 217-230, Jan. 2019, doi: 10.1109/JSSC.2018.2880918.

[3] J. Wang et al., "A 28-nm Compute SRAM With Bit-Serial Logic/Arithmetic Operations for Programmable In-Memory Vector Computing," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, pp. 76-86, Jan. 2020, doi: 10.1109/JSSC.2019.2939682.

[4] J. Zhang, Z. Wang and N. Verma, "In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 4, pp. 915-924, April 2017, doi: 10.1109/JSSC.2016.2642198.

[5] X. Si et al., "A Dual-Split 6T SRAM-Based Computing-in-Memory Unit-Macro With Fully Parallel Product-Sum Operation for Binarized DNN Edge Processors," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 11, pp. 4172-4185, Nov. 2019, doi: 10.1109/TCSI.2019.2928043.

[6] X. Qiao *et al*., "A 65 nm 73 kb SRAM-Based Computing-In-Memory Macro With Dynamic-Sparsity Controlling," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 6, pp. 2977-2981, June 2022, doi: 10.1109/TCSII.2022.3162017.

[7] X. Si et al., "A Local Computing Cell and 6T SRAM-Based Computing-in-Memory Macro With 8-b MAC Operation for Edge AI Chips," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 9, pp. 2817-2831, Sept. 2021, doi: 10.1109/JSSC.2021.3073254.

[8] J. Song, X. Tang, X. Qiao, Y. Wang, R. Wang and R. Huang, "A 28 nm 16 Kb Bit-Scalable Charge-Domain Transpose 6T SRAM In-Memory Computing Macro," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 5, pp. 1835-1845, May 2023, doi: 10.1109/TCSI.2023.3244338.

[9] S. Jeloka, N. B. Akesh, D. Sylvester and D. Blaauw, "A 28 nm Configurable Memory (TCAM/BCAM/SRAM) Using Push-Rule 6T Bit Cell Enabling Logic-in-Memory," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1009-1021, April 2016, doi: 10.1109/JSSC.2016.2515510.

[10] A. Agrawal, A. Jaiswal, C. Lee and K. Roy, "X-SRAM: Enabling In-Memory Boolean Computations in CMOS Static Random Access Memories," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 12, pp. 4219-4232, Dec. 2018, doi: 10.1109/TCSI.2018.2848999.

[11] F. Wei, X. Cui, S. Zhang and X. Zhang, "An 11T SRAM Cell for the Dual-Direction In-Array Logic/CAM Operations," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 4, pp. 2329-2333, April 2024, doi: 10.1109/TCSII.2023.3337119.

[12] Z. Lin et al., "Two-Direction In-Memory Computing Based on 10T SRAM With Horizontal and Vertical Decoupled Read Ports," *IEEE Journal of Solid-State Circuits*, pp. 1-1, 2021, doi: 10.1109/JSSC.2021.3061260.

[13] Z. Lin et al., "In-Memory Computing With Double Word Lines and Three Read Ports for Four Operands," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 5, pp. 1316-1320, May 2020, doi: 10.1109/TVLSI.2020.2976099.

[14] Q. Dong et al., "A 4 + 2T SRAM for Searching and In-Memory Computing With 0.3-V $V_{\mathrm {DDmin}}$," *IEEE Journal of Solid-State Circuits*, vol. 53, no. 4, pp. 1006-1015, 2018, doi: 10.1109/JSSC.2017.2776309.

[15] J. Chen, W. Zhao, Y. Wang, Y. Shu, W. Jiang, and Y. Ha, "A Reliable 8T SRAM for High-Speed Searching and Logic-in-Memory Operations," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 6, pp. 769-780, 2022, doi: 10.1109/TVLSI.2022.3164756.

[16] F. Wei, X. Cui and S. Zhang, "An In-Array Build-In Self-Test Scheme for Embedded SRAM Array," *IEEE Transactions on Circuits and Systems II: Express Briefs*, Early Access Article, doi: 10.1109/TCSII.2024.3375637.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:19 UTC from IEEE Xplore. Restrictions apply.